



**Programación de Estructuras de Datos y Algoritmos Fundamentales**

**Actividad Integradora 2 Conceptos básicos y algoritmos fundamentales**

Farid Gabriel Velasco Martínez A01736669

**07 de abril de 2025**

# Índice

|                                                                             |          |
|-----------------------------------------------------------------------------|----------|
| <b>Índice</b>                                                               | <b>2</b> |
| <b>Introducción</b>                                                         | <b>3</b> |
| <b>Beneficios de las listas doblemente ligadas</b>                          | <b>3</b> |
| <b>Complejidad de las operaciones básicas en la lista doblemente ligada</b> | <b>3</b> |
| Inserción:                                                                  | 3        |
| Borrado:                                                                    | 4        |
| Búsqueda:                                                                   | 4        |
| Impacto:                                                                    | 4        |
| <b>Retos de la implementación de los métodos de ordenamiento y búsqueda</b> | <b>5</b> |
| <b>Conclusión</b>                                                           | <b>5</b> |
| <b>Referencias:</b>                                                         | <b>6</b> |

## Introducción

Continuando con la situación problema planteada, en esta segunda actividad integradora atacamos la problemática desde otro enfoque. Usamos otro tipo de estructuras de datos lineales, listas doblemente ligadas, para almacenar los registros de intentos de ataque y poder implementar diferentes algoritmos de ordenamiento y procesamiento que se desempeñan especialmente bien en este tipo de estructuras.

El volumen de datos que recibimos es masivo por lo que se hace indispensable tener la capacidad de manejar tal volumen eficientemente y sin mucha carga tanto computacional como de memoria. Las listas doblemente ligadas son especialmente adecuadas para ello, ya que aprovechan los punteros para asignar memoria dinámicamente, además de que al agregar elementos en ellas no se realizan operaciones costosas de copia, relocación o transferencia de todos los datos que las componen, sino que simplemente se asigna un nuevo espacio de memoria arbitrario a un nuevo nodo, al que se puede acceder desde otro nodo conectado.

## Beneficios de las listas doblemente ligadas

La estructura de las listas doblemente ligadas nace de una lista ligada simple, la única diferencia es que esta última carece de un puntero en cada nodo que apunta al nodo anterior. Esta simple adición convierte a las listas doblemente ligadas en una estructura de datos mucho más robusta y versátil, por ejemplo, mientras que obtener la información en X elemento en una lista ligada simple tendrá en el peor caso una complejidad de  $O(n)$ , aprovechando las características de una lista doblemente ligada podemos reducir esta complejidad a  $O(\frac{n}{2})$  (cuando la lista está desordenada). A simple vista esta diferencia no parece gran cosa pero implica que la complejidad general de sus métodos será siempre menor que la de una lista ligada simple.

## Complejidad de las operaciones básicas en la lista doblemente ligada

### Inserción:

Esta operación se basa en la búsqueda secuencial de un nodo, método que para usos internos de la estructura denominamos 'get\_dir\_at'. Este método se aprovecha del doble enlazamiento para reducir la complejidad como se mencionó anteriormente a  $O(\frac{n}{2})$ .

Posteriormente enlaza el nuevo nodo con los nodos de alrededor con operaciones simples de asignación, proceso que lógicamente es mucho más simple que en una lista ligada simple, aunque sin beneficio computacional o temporal aparente.

Al terminar nuestra implementación de la lista, nos dimos cuenta que es posible optimizar aún más el desempeño de este y otros métodos que dependen de la búsqueda de un nodo internamente al considerar el estado ordenado de la lista, en este punto sería posible aplicar Binary Search o algún otro algoritmo de búsqueda para llegar al nodo deseado en  $O(n \log n)$ , sin embargo en esta entrega decidimos omitirlo hasta poder hacer un análisis más profundo de los beneficios en cuanto a carga computacional y complejidad temporal que esto supondría, o si es que en realidad no hay diferencia.

### Borrado:

Se podría considerar la operación contraria a la inserción, de igual manera se usa 'get\_dir\_at' con una complejidad de  $O(\frac{n}{2})$  para encontrar el nodo deseado, se enlazan los nodos de alrededor entre sí y una vez que el nodo deseado esté separado se borra. De igual manera que en la inserción, este proceso es más simple que en una lista ligada simple.

### Búsqueda:

Nuestra lista contiene 2 tipos de búsqueda, búsqueda secuencial y búsqueda binaria. La primera es usada cuando la lista no está ordenada, cuando no hay otra opción más que validar nodo por nodo hasta encontrar el indicado, sin embargo la búsqueda binaria se puede emplear cuando la lista está ordenada.

La búsqueda binaria en listas degrada su complejidad temporal a  $O(n)$ , debido a que es un algoritmo especializado para vectores donde el acceso por nodo tiene una complejidad  $O(1)$ . Es cierto que en algunas circunstancias puede desempeñarse mejor que la búsqueda secuencial pero aun en el peor caso y en el caso promedio ambos tipos de búsqueda tendrán una complejidad  $O(n)$ .

## **Impacto:**

Las listas en general, son adecuadas para reducir la carga computacional y de memoria a la hora de generar y almacenar objetos en ellas, evitan copias costosas de los datos como hacen los vectores cuando el espacio reservado es excedido, sin embargo son más difícilmente manipulables en términos de complejidad temporal.

Con las optimizaciones que realizamos en nuestros métodos de la lista doblemente ligada considero que conseguimos una complejidad temporal aceptable y útil para los fines de esta situación problema, sin embargo nos quedamos con el dilema sobre si vale la pena sacrificar eficiencia espacial por eficiencia temporal o viceversa.

## **Retos de la implementación de los métodos de ordenamiento y búsqueda**

Las listas doblemente ligadas, al ser una estructura de datos menos flexible cuando se trata de manipular o acceder sus elementos, implica que se tiene que incorporar lógica más compleja en sus métodos para compensar la ineficiencia temporal que supondría no hacerlo.

En cuanto al algoritmo de ordenamiento implementado, Merge Sort, nos decantamos por su versión iterativa ya que tiene una complejidad computacional menor, el árbol de recursión que se genera con su versión recursiva para ordenar este volumen de datos y otros más grandes simplemente es muy grande y genera el riesgo de provocar un stack overflow. Esta versión iterativa del algoritmo contiene una lógica algo más compleja pero que vale la pena implementar. Por otro lado la implementación de Binary Search como algoritmo de búsqueda fue más difícil al carecer de un acceso aleatorio a los elementos de bajo costo como es el caso en los vectores. Este problema se pudo solucionar al implementar otro método especializado en buscar los elementos al medio de la lista con la menor complejidad temporal posible pero que aun así degrada Binary Search a una complejidad de  $O(n)$ .

## **Conclusión**

Las listas doblemente ligadas son adecuadas cuando se trabaja con grandes volúmenes de datos, pero hay que ser especialmente cuidadosos en cómo estas son manejadas; no se puede implementar cualquier algoritmo de búsqueda u ordenamiento convencional ya que lo más probable es que siempre se degraden a  $O(n)$  o incluso peor. Para obtener un buen desempeño del programa hay que conocer a fondo las características de la estructura de datos que se esté

utilizando y utilizar métodos especializados para trabajar eficientemente en ella.

En el caso de las listas, siempre van a sacrificar eficiencia temporal por eficiencia espacial, pero aquí queda a criterio de los programadores decidir cuál es la estructura de datos más adecuada para una situación específica. En este caso si el volumen de datos llega a ser mucho mayor, la eficiencia espacial toma más peso y me decantaría por usar listas para almacenar la información, sin embargo ante la situación actual, considero que usar vectores sigue siendo un enfoque más viable, ya que no solo se requiere almacenar la información, sino también analizarla tan rápido como sea posible para responder a tiempo a las amenazas que se presenten.

## Referencias:

1. GeeksforGeeks. (2024, 14 octubre). *C++ program for binary Search*. GeeksforGeeks.  
[https://www.geeksforgeeks.org/cpp-binary-search/?ref=header\\_outind](https://www.geeksforgeeks.org/cpp-binary-search/?ref=header_outind)
2. GeeksforGeeks. (2025, enero 29). *Merge Sort Data Structure and Algorithms Tutorials*. GeeksforGeeks. <https://www.geeksforgeeks.org/merge-sort/>
3. GeeksforGeeks. (2025, febrero 19). *Doubly linked list tutorial*. GeeksforGeeks.  
[https://www.geeksforgeeks.org/doubly-linked-list/?ref=header\\_outind](https://www.geeksforgeeks.org/doubly-linked-list/?ref=header_outind)
4. GeeksforGeeks. (2025, febrero 20). *Iterative merge sort*. GeeksforGeeks.  
<https://www.geeksforgeeks.org/iterative-merge-sort/>